

Project 23 - Reinforcement Learning: Model-Based Control on CartPole

Presented by: Kyrillus Mehanni, Feodor Razumovich

Why model-based RL?

Model-free RL can learn strong policies, but typically needs a very large number of real environment interactions. Model-based control instead learns the environment's dynamics, then plans against that model - which can be far more sample-efficient.

Model-free (e.g. DQN, policy gradients)

- Learns a policy / value function from reward
- Each improvement needs fresh environment steps
- Sample-hungry: often millions of steps
- No world model to plan ahead with

Model-based (our approach)

- Learns a dynamics model $\hat{s}_{t+1} = \hat{f}\theta(s_t, a_t)$
- The model can be queried thousands of times for free
- Planning selects actions without extra real steps
- Much better sample efficiency on control tasks

Problem Statement

We treat CartPole-v1 as a control problem and ask how efficiently it can be solved from interaction alone.

Goal

Keep the pole balanced on the moving cart for as long as possible, i.e. maximise the episode return (up to 500 per episode).

Core Challenge

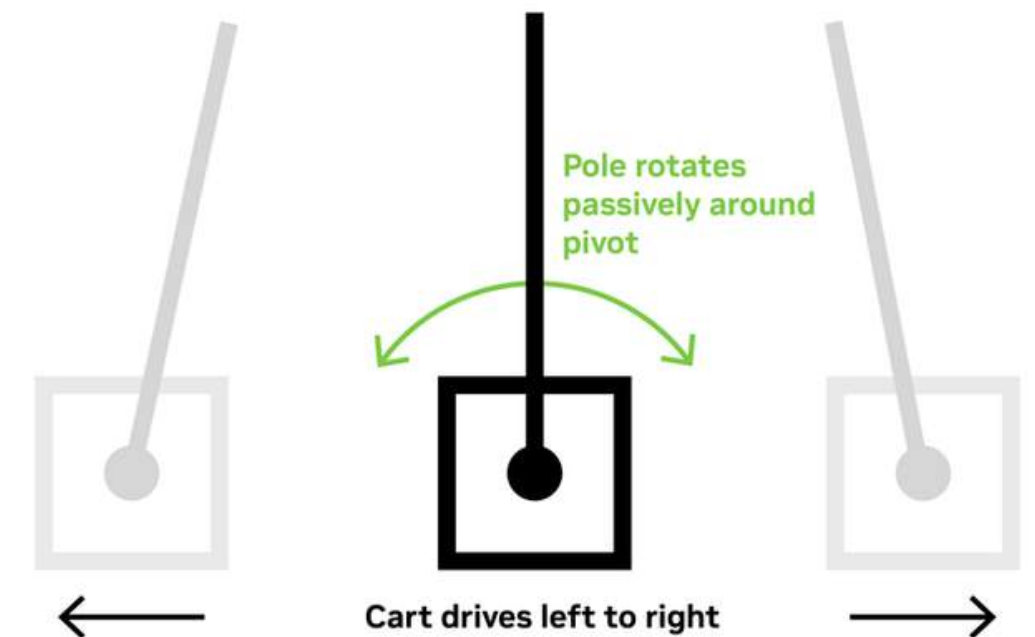
Do this from as few real environment interactions as possible. Sample efficiency is the property we care about most.

Setting

A 4-dimensional continuous state and 2 discrete actions, with a reward of +1 per surviving step and no prior knowledge of the physics.

Success criterion

Average return of at least 475 over 100 episodes, reported against the number of real environment steps used.



Example Cartpole Environment

The CartPole-v1 environment

Observation space: 4 continuous variables

#	Observation	Range
0	Cart position	$[-4.8, 4.8]$
1	Cart velocity	$(-\infty, \infty)$
2	Pole angle	$[-0.418, 0.418]$ rad ($\pm 24^\circ$)
3	Pole angular velocity	$(-\infty, \infty)$

Reward: +1 for every step the pole stays up →
maximising return = staying alive longer.

Actions

2 discrete: push cart left (0) or right (1).

Episode ends when

$|\text{pole angle}| > 12^\circ$, $|\text{cart pos}| > 2.4$, or 500 steps reached.

Solved

average return ≥ 475 over 100 episodes (max possible = 500).

Our approach



Periodically aggregate the newly visited data and retrain $\hat{f}_\theta$.

The dynamics model

What it predicts

- An MLP maps (state, action) $\rightarrow$ next state.
- We predict the state delta $\Delta s_t = s_{t+1} - s_t$ rather than s_{t+1} directly; deltas are smaller and easier to learn (Nagabandi et al., 2018).
- Reconstruct the prediction as $\hat{s}_{t+1} = s_t + \Delta \hat{s}_t$.
- Inputs/targets are normalised so all 4 state dimensions contribute on a comparable scale.

How it is trained

- Supervised regression with mean-squared-error loss on observed transitions.
- Standard train / validation split; validation loss drives early stopping and hyper-parameter choices.
- Discrete actions (0/1) are fed as a simple encoded input feature.
- Output: a fast, differentiable surrogate of the true CartPole physics.

Planning with Model Predictive Control

MPC turns a learned model into a controller by re-planning at every step:

1 · Imagine

From the current state, roll candidate action sequences of horizon H forward through $\hat{f}_\theta$.

2 · Score

Sum the predicted reward (and a cost keeping the pole upright / cart centred) over each rollout.

3 · Act

Execute only the first action of the best sequence on the real environment.

4 · Replan

Observe the true next state and repeat. This receding-horizon scheme corrects model errors at every step.

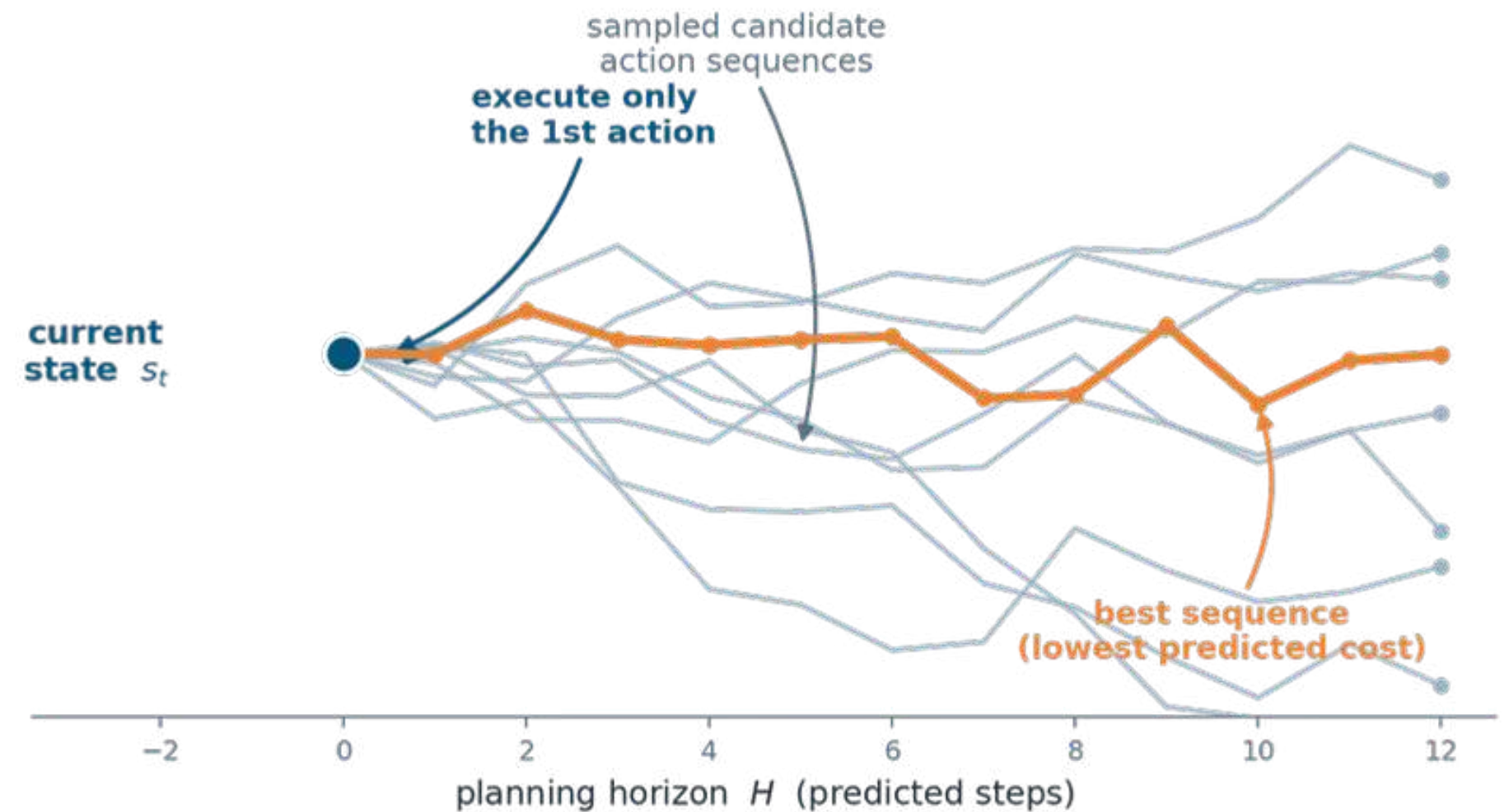
Choosing actions: random shooting vs. CEM

Random shooting

- Sample N random action sequences.
- Evaluate all under the model, pick the highest-scoring one.
- Simple, fully parallel, and a strong baseline.

Cross-Entropy Method (CEM)

- Sample, keep the top 'elite' sequences, refit the sampling distribution toward them, repeat.
- Iteratively focuses the search on promising actions, typically matching the same quality with fewer samples.



Random shooting: sample many sequences, keep the best.

Scoring rollouts and collecting data

Scoring a rollout

- Reward is +1 per surviving step, so a rollout's score is essentially how long it stays upright.
- An optional cost on |pole angle| and |cart position| favours balanced, centred states.
- Termination must be respected: apply CartPole's known end-rule to predicted states, or learn a 'done' predictor.

Collecting the data

1. Random warm-up to cover the basic dynamics.
2. Train $\hat{\theta}$ on the data (MSE, train / val split).
3. Control with MPC; it visits new, relevant states.
4. Aggregate the new data and retrain.

How we measure success

Evaluation protocol

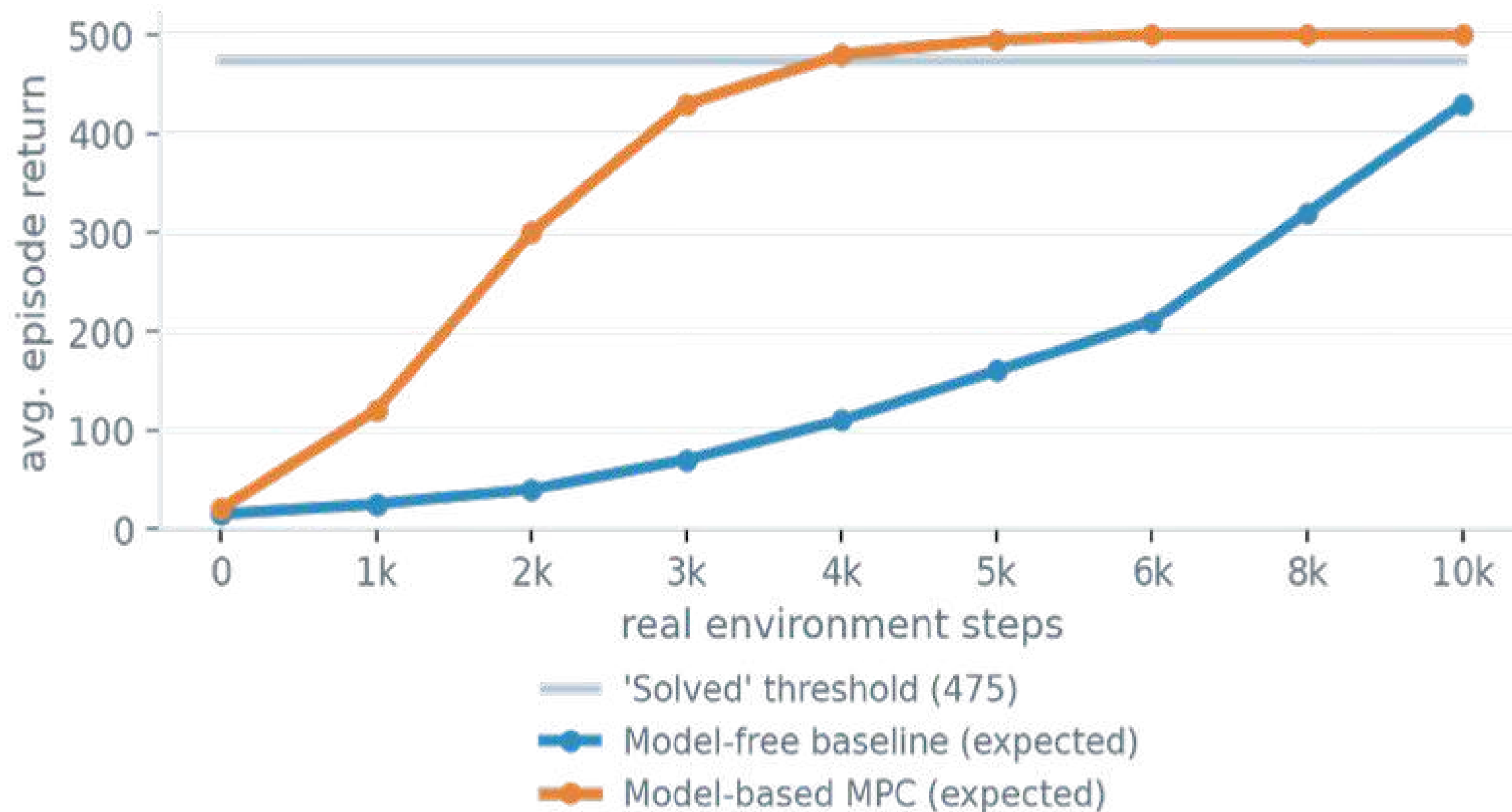
What we report

- Average episode return vs. number of real environment steps (the learning curve).
- Evaluate on a fixed set of seeds not used during data collection.
- Average return of at least 475 over 100 episodes.

Planned ablations

- Planning horizon H (e.g. 10 / 20 / 30).
- Number of sampled action sequences N .
- Random shooting vs. CEM.
- Predicting Δs vs. absolute next state.
- Dynamics-model size / training data amount.

Expected results



Hypotheses

- Model-based MPC reaches high return in far fewer real steps than a model-free baseline.
- A longer horizon H helps, up to the point where model error compounds and hurts.
- CEM matches random shooting with fewer sampled sequences.

References

- [1] Nagabandi, A., Kahn, G., Fearing, R. S., & Levine, S. (2018). Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning. ICRA. [arXiv:1708.02596](#)

- [2] Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS). NeurIPS. [arXiv:1805.12114](#)

- [3] Towers, M. et al. Gymnasium. CartPole-v1 (classic control). [gymnasium.farama.org](#)